Tutorials

A Practical Guide to Spec-Driven Development

Copy page

Spec-Driven Development (SDD) is a methodology that transforms how we work with AI coding agents by treating them as literal-minded but highly capable pair programmers who excel when given explicit, detailed instructions.

This guide uses a real-world example throughout – building a production-ready notification system – to demonstrate the dramatic difference between traditional prompting and spec-driven development. By the end, you'll understand not just the "what" but the "how" and "why" of this powerful approach.

The Evolution: From Vibe-Coding to Systematic Development

The Problem with Traditional Prompting

When AI coding agents first emerged, we treated them like search engines – type a query, get code back. This "vibe-coding" approach works great for quick prototypes, but it breaks down when building serious, mission-critical applications.

Consider this typical interaction:

Traditional Prompting Cycle (Click to Expand)

Each iteration loses context from previous discussions. The agent makes reasonable assumptions that turn out wrong. You spend more time correcting course than building features.

zencoder

›

Traditional Prompting

- Iterative discovery
- Multiple corrections needed
- Context loss over time
- Generic implementations
- Unpredictable results

Spec-Driven Development

- Upfront clarity
- Single source of truth
- Persistent context
- Tailored implementations
- Predictable outcomes

Why SDD Works in Production Environments

In established codebases, simple prompting often produces code that technically works but doesn't fit. The AI might choose different state management than your existing patterns, recreate functionality that already exists, or miss compliance requirements that aren't explicitly stated.

Spec-driven development addresses this by front-loading context. Your specification becomes the complete picture—not just what to build, but how it should integrate with existing systems, what patterns to follow, and what constraints to respect. This is especially critical in brownfield environments where implicit knowledge matters as much as explicit requirements.

The Four Phases of Spec-Driven Development

1 **Specify**

zencoder

›

Create a technical architecture that respects your existing codebase and constraints

3 **Tasks**

Break down the implementation into discrete, testable units of work

4 **Implement**

Execute against the plan with continuous validation against specifications

Let's walk through each phase using a real example: building a production-ready notification system for an e-commerce platform.

Our Example: Real-Time Notification System

Throughout this guide, we'll build a notification system with these requirements:

- Multi-channel delivery (email, SMS, in-app)
- User preference management
- Retry logic with exponential backoff
- Delivery tracking and analytics
- Real-time updates

Phase 1: Specify – Creating Your North Star

The specification is your contract with the AI agent. It defines success in user terms, not technical jargon.

zencoder

```
- User story and stakeholders
- Measurable success criteria
- Functional and non-functional requirements
- Explicit constraints (what NOT to build)
- Technical context and integration points
- Acceptance tests

Be specific enough that another developer could implement without clarification.
```

Traditional Approach

```
"Build a notification system for order updates"
```

This seems clear to you because you have context. But the AI agent must guess at dozens of implicit requirements.

Spec-Driven Approach

Specification | Why This Works

zencoder

›

As a customer, I want to receive timely updates about my order status through my

Stakeholders
- **Primary**: E-commerce customers (50K daily active users)
- **Secondary**: Customer service team (reduce support tickets by 40%)
- **Tertiary**: Operations team (monitor delivery performance)

Success Criteria
1. **Delivery Speed**: 95% of notifications delivered within 60 seconds
2. **Reliability**: 99.9% successful delivery rate (after retries)
3. **Preference Compliance**: 100% adherence to user channel preferences
4. **Support Impact**: 40% reduction in "where's my order" tickets

Functional Requirements

Multi-Channel Support
- Email notifications via SendGrid
- SMS notifications via Twilio
- In-app notifications via WebSocket
- Each channel independently toggleable

Notification Types
1. Order confirmed
2. Payment processed
3. Order shipped (with tracking)
4. Out for delivery
5. Delivered
6. Delivery failed
7. Refund initiated
8. Refund completed

User Preferences
- Global on/off switch
- Per-channel toggles
- Per-notification-type preferences
- Quiet hours (no SMS between 10 PM - 8 AM)

zencoder

›

```
- Different strategies per channel
- Dead letter queue after final failure

### Analytics Requirements
- Track delivery rate per channel
- Track open/click rates for email
- Track user preference changes
- Monitor retry patterns
- Alert on delivery degradation

## Non-Functional Requirements

### Performance
- Handle 10,000 concurrent notifications
- 60-second end-to-end delivery SLA
- No more than 100ms API response time

### Security
- PII data encryption at rest
- Secure token generation for unsubscribe links
- Rate limiting per user (max 50 notifications/day)

### Compliance
- CAN-SPAM compliant unsubscribe mechanism
- TCPA compliance for SMS
- GDPR-compliant data handling

## Explicit Constraints (DO NOT)
- Do NOT implement push notifications (Phase 2)
- Do NOT build a custom email service (use SendGrid)
- Do NOT modify the existing User model
- Do NOT add social media notifications
- Do NOT implement notification templates editing UI (admins will use code)
- Do NOT create a separate notification service (embed in monolith)
```

zencoder

›

```
## Acceptance Tests
1. User can enable/disable individual channels
2. Notification arrives within 60 seconds of trigger
3. Failed notifications retry with correct backoff
4. Quiet hours are respected for SMS
5. Unsubscribe link works without authentication
6. Analytics dashboard shows real-time metrics
```

Pro Tip: Your specification should be detailed enough that another developer could implement it without asking clarifying questions. If you find yourself explaining things during implementation, add those clarifications to the spec for next time.

Phase 2: Plan – From Vision to Architecture

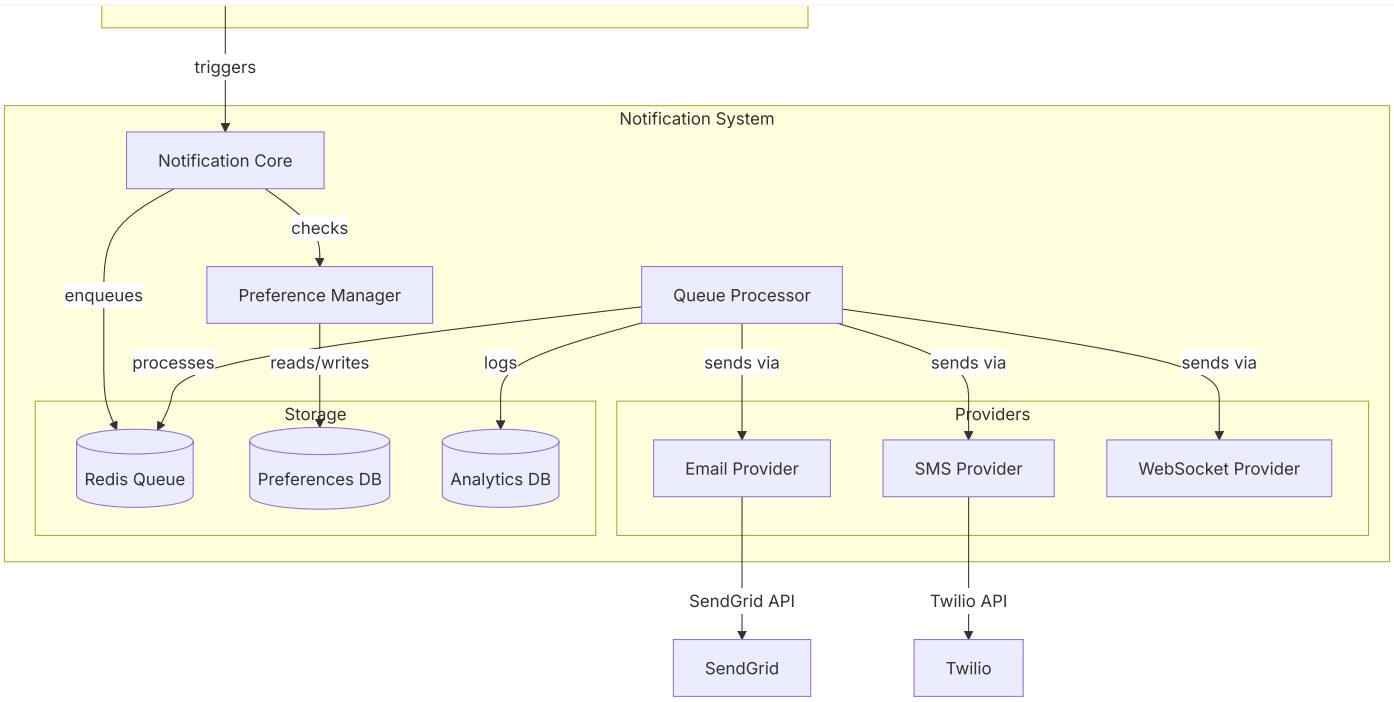
With a clear specification, your AI agent can create a comprehensive technical plan that considers your entire ecosystem.

Setting Up the Planning Phase

In Zencoder, you'll leverage the **Coding Agent** with **Repo Grokking™** to understand your existing codebase:

```
Based on the specification above, create a technical implementation plan.
Consider our existing services and patterns.
Identify integration points and potential conflicts.
```

The Generated Plan



Notice how the plan respects our constraints: no push notifications, uses existing services, maintains API versioning.

Phase 3: Tasks – Breaking Down the Mountain

With architecture defined, we decompose the implementation into manageable, testable chunks.

Generating the Task List

```
Break down the implementation into discrete tasks.
Each task should be:
- Independently implementable
- Testable in isolation
- Reviewable as a single PR
```

Core Models
Preference Manager
Provider Implementations
Queue System
Integration Layer
Analytics & Monitoring
Testing Suite

Using Zencoder: Each task becomes a conversation with your **Coding Agent**. The agent maintains context across tasks through the specification and plan, ensuring consistency.

Phase 4: Implement – Guided Execution with Validation

Now we implement each task, with the AI agent having full context of specifications, architecture, and dependencies.

With specifications defining requirements, tests providing guardrails, and a technical plan establishing architecture, Phase 4 implementation becomes significantly more autonomous. The agent can execute with confidence while you shift attention to planning the next feature. Verification catches deviations automatically, eliminating the need for constant supervision.

Traditional vs. Spec-Driven Implementation

Let's implement the EmailProvider to see the difference:

zencoder

›

```
human: Implement an email notification provider
```

The agent might generate:

```
class EmailProvider {
  async send(to: string, subject: string, body: string) {
    // Generic implementation
    await sendEmail(to, subject, body);
  }
}
```

This is technically correct but misses:

- Error handling strategy
- Retry logic integration
- Analytics tracking
- Template system
- Unsubscribe links
- Compliance requirements

The spec-driven implementation includes everything from the specification:

- Compliance features (unsubscribe links, CAN-SPAM)
- Analytics tracking
- Retry logic coordination
- Error classification
- Template system
- Health monitoring

zencoder

›

Verification: The Industrial-Grade Safety Net

Specifications without verification are just documentation. The power of spec-driven development comes from continuous validation:

Test-Driven Guardrails Each acceptance criterion in your specification becomes a test case. As the agent implements:

1. Tests validate correctness against spec requirements
2. Failures indicate deviation from specifications
3. Success confirms alignment with intended behavior

This verification layer transforms AI coding from “hope it works” to “prove it works.”

Example from our notification system:

zencoder

```
    const p95 = percentile(deliveryTimes, 95);
    expect(p95).toBeLessThan(60000); // 60 seconds
  });
});

// Specification says: "Respect quiet hours for SMS"
describe('SMSProvider quiet hours', () => {
  it('queues SMS during quiet hours', async () => {
    setTime('23:00'); // 11 PM
    await notificationService.trigger(smsNotification);
    expect(immediatelySent).toBe(false);
    expect(queuedFor).toBe('08:00');
  });
});
```

Without these guardrails, the agent might implement retry logic that “looks right” but uses the wrong intervals, or add quiet hours that check the wrong timezone.

Implementing in Zencoder: Practical Workflow

Now let’s set up your Zencoder environment for spec-driven development.

Step 1: Organize Your Specifications

Create a dedicated structure for your specifications:

zencoder

```
│   │   ├── authentication.md
│   │   └── payment-processing.md
│   ├── plans/              # Technical plans
│   │   └── notifications-plan.md
│   └── rules/              # Zen Rules for consistency
│       └── sdd-standards.md
```

Step 2: Configure Zen Rules for SDD

Create a Zen Rules file to enforce spec-driven patterns:

zencoder

›

```
# Spec-Driven Development Standards

## Before Implementation
1. Always check for an existing specification in `.zencoder/specs/`
2. If no spec exists, request one before proceeding
3. Validate implementation plans against specifications

## During Implementation
1. Each file should reference its governing specification
2. Use specification acceptance criteria as test cases
3. Flag any deviations from spec as "SPEC_DEVIATION: [reason]"

## Validation Checklist
- [ ] Implementation matches specification requirements
- [ ] All explicit constraints (DO NOTs) are respected
- [ ] Acceptance criteria have corresponding tests
- [ ] Performance requirements are validated
- [ ] Security requirements are implemented
```

Step 3: Initialize Your Agent Workflow

1 Generate Project Context

Use the **Repo-Info Agent** to create comprehensive project context:

```
/repo-info
Generate a comprehensive analysis including architecture patterns,
dependencies, and coding conventions
```

zencoder

```
Here's the specification for our notification system:
[paste specification]

Please review and identify any clarifications needed before we proceed.
```

3 Generate the Plan

```
Based on the specification and our codebase analysis, create a
technical implementation plan including architecture, database
schema, and integration points.
```

4 Create Task List

```
Break down the implementation into discrete, testable tasks.
Each task should be a single PR worth of work.
Include time estimates and dependencies.
```

zencoder

```
Implement [Task Name] according to the specification and plan.
Reference: `.zencoder/specs/notifications.md` section [X]
Acceptance criteria: [list from spec]
```

Step 4: Leverage Multi-Agent Collaboration

Unit Testing Agent

Generate tests directly from specifications:

```
Generate unit tests for EmailProv
- Notifications delivered within
- Failed sends retry with exponen
- Unsubscribe links included in a
```

Link to Unit Testing Agent

E2E Testing Agent

Validate complete user flows:

```
Create E2E tests for the notifica
1. User disables SMS notificatior
2. Order status changes
3. Verify only email and in-app r
```

Link to E2E Testing Agent

Working Across Multiple Sessions

Spec-driven development truly shines in long-running tasks that span multiple coding sessions.

State Management Strategy

zencoder

›

```
# Notification System Implementation Progress

## Completed Tasks
- [x] Database migrations (PR #234)
- [x] Core models (PR #235)
- [x] PreferenceManager service (PR #236)
- [x] EmailProvider implementation (PR #237)

## Current Task
- [ ] SMSProvider implementation
  - Twilio client configured
  - Basic send method complete
  - TODO: Add retry logic
  - TODO: Add quiet hours check

## Upcoming Tasks
- [ ] WebSocketProvider
- [ ] Queue processor
- [ ] Analytics dashboard

## Blockers
- Waiting for Twilio API credentials from DevOps

## Notes for Next Session
- SMS provider needs special handling for international numbers
- Consider adding provider circuit breaker pattern
- Performance test results: current implementation handles 5K/sec
```

Better Code Review Through Structure

Spec-driven development fundamentally improves code review. Instead of reviewing scattered changes across multiple files and reverse-engineering the developer's intent,

zencoder

›

- Unclear which changes address which requirements
- Reviewer must mentally reconstruct the feature
- Easy to miss edge cases or requirement deviations

Spec-Driven PR:

- Each PR maps to a specific task from the breakdown
- Task links to relevant specification section
- Reviewer validates: "Does this implementation satisfy the spec requirements?"
- Tests prove acceptance criteria are met

This top-down review process—specification → plan → task → implementation—keeps humans meaningfully in the loop while AI handles execution.

Model Selection Matters: Different AI models have different strengths. For example, we've found that Claude Sonnet 4.5 excels at state management and maintaining context across long sessions. Experiment with different models for different phases of development. **Read our Sonnet 4.5 review** to learn more about model-specific capabilities.

Resuming Work

When starting a new session:

```
Review the progress in progress.md and test-status.json.
Check git log for recent changes.
Continue with the current task, maintaining consistency with completed work.
```

zencoder

›

When to Use Each Approach

Not every coding task needs full specification. Here's your decision framework:

Use Traditional Prompting

Perfect for:

- Quick prototypes and experiments
- Simple utility functions
- Bug fixes with clear solutions
- Learning and exploration
- One-off scripts
- Small UI tweaks

Example tasks:

- "Fix the sorting bug in the user table"
- "Add a loading spinner to the form"
- "Create a script to clean up old logs"

Use Spec-Driven Development

Essential for:

- Production features
- Multi-file implementations
- Integration with existing systems
- Features with compliance requirements
- Long-running development tasks
- Team projects requiring consistency

Example tasks:

- "Build a notification system"
- "Implement payment processing"
- "Create a data synchronization service"

Patterns for Success

Pattern 1: Living Specifications

Your specifications should evolve with your understanding:

zencoder

```
- Clarified retry backoff intervals
- Added language preference support

## v1.1.0
- Added analytics requirements
- Specified performance targets

## v1.0.0
- Initial specification
```

Pattern 2: Specification Templates

Create templates for common features:

- API Feature Template
- Background Job Template

Pattern 3: Test-Driven Specifications

Include test scenarios in your specifications:

zencoder

```
When: order status changes to "shipped"
Then: Email, SMS, and in-app notifications sent within 60 seconds

### Edge Case: Quiet Hours
Given: Current time is 11 PM, user has SMS enabled
When: Order delivered notification triggered
Then: Email and in-app sent immediately, SMS queued for 8 AM

### Error Case: Provider Failure
Given: SendGrid API returns 500 error
When: Email notification attempted
Then: Retry after 1 minute, then 5 minutes, then 15 minutes
```

Anti-Patterns to Avoid

Common Pitfalls in Spec-Driven Development

- **Over-Specifying Implementation Details**: Specify what, not how
- **Under-Specifying Edge Cases**: Think through error scenarios
- **Ignoring Existing Patterns**: Respect your codebase conventions
- **Skipping the Planning Phase**: Architecture matters
- **Not Updating Specs**: Keep them living documents

Establishing Reproducible Development Processes

One challenge with AI-assisted development is the wide variation in individual developers' prompting skills and AI experience. A senior developer who knows how to prompt effectively might achieve 3x productivity, while a junior developer struggles with the same tools.

Spec-driven development creates a standardized process that works regardless of individual AI expertise. The methodology is embedded in the specifications and workflow,

- Code quality remains consistent because requirements and verification are explicit
- Knowledge lives in specifications and tests, not in individuals' prompting techniques
- Teams can scale AI-assisted development without scaling AI expertise

The result is a unified approach to development that produces predictable, high-quality outcomes across your entire engineering organization.

Getting Started: Your First Spec-Driven Feature

Ready to try spec-driven development? Start with a small, self-contained feature:

1 Choose a Feature

Pick something with:

- Clear user value
- 3-5 day implementation
- Minimal external dependencies
- Defined success criteria

2 Write Your First Specification

Use this minimal template:

zencoder

›

```
## Success Criteria
1. [Measurable outcome]
2. [Measurable outcome]

## Requirements
- [Functional requirement]
- [Functional requirement]

## Constraints
- Do NOT [constraint]
- Must use [existing system]
```

3 Generate Plan with Zencoder

Open **Coding Agent**:

```
Create a technical plan for this specification.
Consider our existing codebase and patterns.
```

4 Implement and Iterate

Work through tasks systematically, updating your specification as you learn.

Looking Forward

Spec-driven development isn't just about writing better prompts—it's about fundamentally changing how we collaborate with AI coding agents. By treating them as highly capable but literal-minded partners who excel with clear direction, we unlock their true potential.

zencoder

›

- Better alignment with requirements
- More maintainable code
- Comprehensive test coverage
- Living documentation

As AI agents become more sophisticated, the ability to precisely specify what we want becomes even more valuable. The agents gain capabilities, but they still need clear direction to apply those capabilities effectively.

> **Remember**: Spec-driven development and traditional prompting aren't mutually exclusive. Use the right tool for the job. Quick fixes and explorations benefit from conversational prompting, while production features deserve the rigor of specifications.

‹ Contributing to Open Source with OSS Contributor Agent

Convert Figma Designs to Code with Zencoder ›